DESIGN DOCUMENT

A) Daily Target Earnings Vs Current Earnings

Product Vision:

Gig economy drivers often set daily earning goals (for example, earning ₹1200 during a shift), but they lack tools that help them understand whether they are progressing toward those goals while working.

Drivers usually rely on intuition to decide:

- whether they are earning fast enough
- whether they should continue driving
- whether they are falling behind their targets

This system provides a **real-time analytics dashboard that monitors driver earnings performance during a shift and predicts whether the driver will meet their goal**.

The system continuously analyzes driver earnings data and provides insights including:

- current earning velocity
- required velocity to meet goals
- predicted final earnings
- progress toward the goal

The goal of the product is to **help drivers understand their shift performance in real time and make better decisions during their working hours**.

2.Core Product Features

Feature 1:- Driver Goal Setting

Drivers can define a **shift goal** specifying:

- target earnings
- target shift duration
- shift start time
- shift end time

These goals define the **earning target the driver intends to reach during the shift**.

Feature 2: Earnings Velocity Tracking

The system calculates the driver's **current earning velocity**, which represents the rate at which the driver is earning money during the shift.

Velocity represents earnings per hour. This value is continuously updated as the driver accumulates earnings during the shift.

Feature 3: Static Target Velocity Calculation

Each driver goal implies a **required earning rate** needed to reach the goal.

Target velocity = Target earnings/ Target Time

This value represents the average speed at which the driver must earn money to achieve the shift goal.

Feature 4: Dynamic Target Velocity Calculation

While the static target velocity provides the average earning rate required across the entire shift, it does not account for how the driver is performing during the shift.

If a driver earns less than expected early in the shift, the required earning rate for the remaining time increases. Similarly, if the driver earns more than expected early in the shift, the required earning rate for the remaining time decreases.

To reflect this behavior, the system calculates a dynamic target velocity that adjusts continuously based on the driver's current progress.

The dynamic target velocity is computed using the remaining earnings required and the remaining time in the shift.

remaining_earnings = target_earnings - current_earnings

remaining_hours = target_hours - elapsed_hours

dynamic_target_velocity = remaining_earnings / remaining_hours

Feature 5: Shift Completion Prediction

The system estimates the driver’s expected final earnings at the end of the shift based on their current velocity. The assumption is that the driver continues at the same current velocity.

remaining_hours = target_hours - elapsed_hours future_earnings = current_velocity × remaining_hours

predicted_final = current_earnings + future_earnings

This prediction helps determine whether the driver is likely to meet their goal.

Feature 6: Performance Forecast Calculation

Based on the predicted earnings, the system classifies driver performance into categories.

Forecast categories:

Condition	Status
Predicted ≥ 110% target	Ahead
Predicted ≥ 90% target	On Track
Predicted < 90% target	At Risk

These categories provide a quick summary of the driver's performance during the shift.

Feature 7: Goal Process Tracking

The system calculates how much of the goal the driver has completed.

progress_percent = (current_earnings / target_earnings) × 100

This value is displayed on the dashboard as a **progress indicator**.

Feature 8: Current Velocity vs Target Velocity Graph

The system records earnings progression throughout the shift, enabling visualization of how driver performance evolves over time.

This timeline allows the dashboard to display earnings growth and velocity changes during the shift.

Why is velocity used as the primary metric?

Velocity is used as the core metric because it provides a simple and interpretable measure of a driver's earning rate during a shift.

$$\text{velocity} = \text{cumulative\_earnings} / \text{elapsed\_hours}$$

It represents how quickly earnings are accumulating over time.This metric allows the system to estimate future earnings by assuming the driver continues earning at a similar rate. Using velocity, the predicted final earnings can be calculated as:

$$\text{predicted\_final} = \text{current\_earnings} + (\text{velocity} \times \text{remaining\_hours})$$

Velocity is well suited for the MVP because it is easy to compute, updates in real time, and provides a clear indicator of whether a driver is on track to meet their earnings goal.

Future Extensions

Possible future improvements include:

- machine learning earnings forecasting
- demand prediction integration
- driver performance recommendations
- adaptive goal suggestions
- long-term driver analytics

These improvements would allow the system to evolve from a basic analytics dashboard to a more advanced decision-support system for drivers.

B)Pulse Scoring and Stress Detection

1. Objective

The objective of this module is to analyze driver behavioral signals using sensor data and identify patterns that may indicate driver stress or abnormal driving environments. The system generates continuous pulse scores for visualization and flags specific stress events for monitoring and safety analysis.

2. Data Inputs

The system uses two main sensor data sources:

Accelerometer Data

accelerometer_data.csv contains

- trip_id
- timestamp
- accel_x
- accel_y

This data represents vehicle motion and sudden movements.

Audio Intensity Data

audio_intensity_data.csv contains

- trip_id
- timestamp
- audio_level_db

- audio_classification
- sustained_duration_sec

This data captures environmental noise and possible arguments or loud conversations inside the vehicle.

3. Pulse Scoring Design

The pulse scoring module generates **continuous behavioral signals** based on motion and audio inputs.

Motion score represents sudden vehicle movement intensity.

magnitude = sqrt $(ax^2 + ay^2)$
motion_score = min (magnitude / 7, 1)
Higher motion values indicate aggressive acceleration or braking.

Audio score combines:

- sound intensity (dB)
- sound classification
- duration of the sound event

The score is computed as:

audio_score = 0.5 * classification_score + 0.3 * loudness_component + 0.2*duration_component
Where classification scores represent different sound categories such as quiet, normal conversation, loud, or argument.

4. Pulse Scoring Pipeline

The output dataset contains:

timestamp
trip_id
driver_id
motion_score
audio_score
This dataset is used by the dashboard to visualize the driver's pulse graph.

5. Stress Detection Design

The stress detection module analyzes motion and audio patterns to identify abnormal events.

Stress events may indicate:

- aggressive driving
- arguments inside the vehicle
- sudden high noise levels

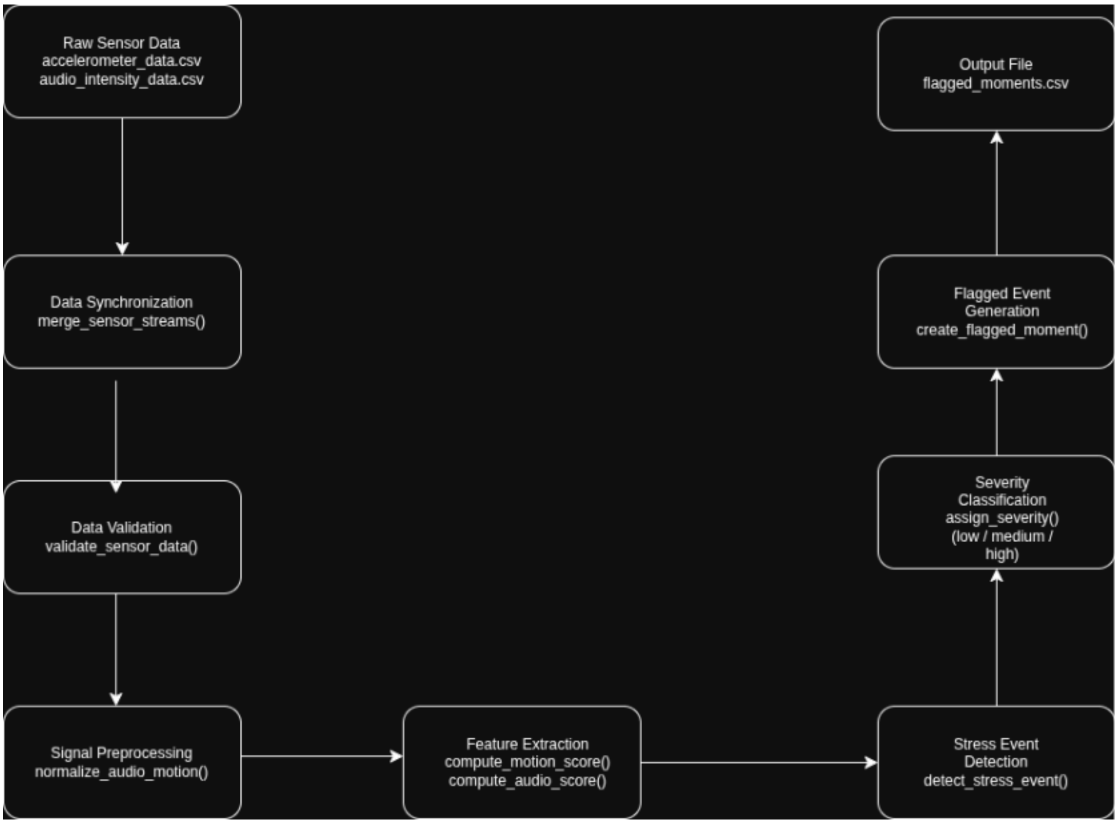
- unsafe driving behavior

Stress events are detected when:

motion_score > threshold or audio_score > threshold
Events are classified into severity levels:

Low
Medium
High
based on score intensity and duration

6. Stress Detection Pipeline



The output dataset contains:

flag_id
driver_id
trip_id
timestamp
flag_type

severity
These flagged events are displayed in the dashboard trip history.

7. System Integration

The generated analytics are integrated into the backend APIs:

Pulse API

GET /pulse/{driver_id}

Returns pulse score timeline for visualization.

Stress API

GET /stress/flags/driver/{driver_id}

Returns detected stress events for a driver.

8. Design Advantages

The design provides several benefits:

- Modular architecture separating pulse scoring and stress detection
- Continuous monitoring through pulse scores
- Event-based analysis through stress detection
- Scalable pipeline architecture
- Efficient API responses using precomputed datasets

SUMMARY

The pulse scoring module generates continuous behavioral signals from motion and audio sensor data, while the stress detection module analyzes these signals to identify abnormal driving or environmental stress events.

Future Extensions:

- Real-time stress monitoring
- Machine learning–based stress detection
- Integration of additional sensors (GPS, gyroscope, biometrics)
- Personalized driver behavior baselines
- Scalable database storage instead of CSV files
- Real-time driver alerts and feedback system